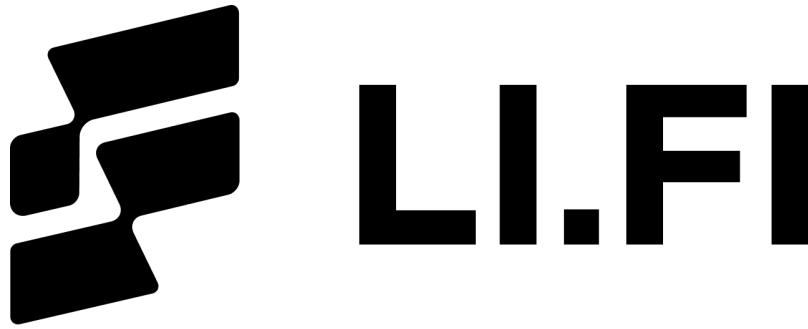




---

## LI.FI Security Review

---

LayerSwapFacet.sol(v1.0.0)

### Security Researcher

Sujith Somraaj ([somraajsujith@gmail.com](mailto:somraajsujith@gmail.com))

Report prepared by: Sujith Somraaj

July 15, 2026

# Contents

|          |                                                                        |          |
|----------|------------------------------------------------------------------------|----------|
| <b>1</b> | <b>About Researcher</b>                                                | <b>2</b> |
| <b>2</b> | <b>Disclaimer</b>                                                      | <b>2</b> |
| <b>3</b> | <b>Scope</b>                                                           | <b>2</b> |
| <b>4</b> | <b>Risk classification</b>                                             | <b>2</b> |
| 4.1      | Impact . . . . .                                                       | 2        |
| 4.2      | Likelihood . . . . .                                                   | 3        |
| 4.3      | Action required for severity levels . . . . .                          | 3        |
| <b>5</b> | <b>Executive Summary</b>                                               | <b>3</b> |
| <b>6</b> | <b>Findings</b>                                                        | <b>4</b> |
| 6.1      | Medium Risk . . . . .                                                  | 4        |
| 6.1.1    | Refunds and swap leftovers may be sent to a proxy or relayer . . . . . | 4        |
| 6.2      | Low Risk . . . . .                                                     | 4        |
| 6.2.1    | Mismatched swap output can drain the Diamond's bridge asset . . . . .  | 4        |
| 6.3      | Gas Optimization . . . . .                                             | 4        |
| 6.3.1    | Compact signatures can reduce calldata costs . . . . .                 | 4        |
| 6.3.2    | Constant EIP-712 values are hashed on every call . . . . .             | 5        |
| 6.3.3    | Calldata signature is unnecessarily copied into memory . . . . .       | 5        |

# 1 About Researcher

Sujith Somraaj is a distinguished security researcher and protocol engineer with over eight years of comprehensive experience in the Web3 ecosystem.

In addition to working as a Lead Security Researcher at Spearbit, Sujith is also the security researcher and advisor for leading bridge protocol LI.FI and also is a former founding engineer and current security advisor at Superform, a yield aggregator with over \$170M in TVL.

Sujith has experience working with protocols / funds including Coinbase, Solana Foundation, Layerzero, Edge Capital, Berachain, Optimism, Ondo, Sonic, Monad, Blast, ZkSync, Decent, Drips, SuperSushi Samurai, DistrictOne, Omni-X, Centrifuge, Superform-V2, Tea.xyz, Paintswap, Bitcorn, Sweep n' Flip, Byzantine Finance, Variational Finance, Satsbridge, Rova, Horizen, Earthfast, Angles and multiple other protocols.

Learn more about Sujith on [sujithsomraaj.xyz](https://sujithsomraaj.xyz) or on [cantina.xyz](https://cantina.xyz)

## 2 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of that given smart contract(s) or blockchain software. i.e., the evaluation result does not guarantee against a hack (or) the non existence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, I always recommend proceeding with several audits and a public bug bounty program to ensure the security of smart contract(s). Lastly, the security audit is not an investment advice.

This review is done independently by the reviewer and is not entitled to any of the security agencies the researcher worked / may work with.

## 3 Scope

- src/Facets/LayerSwapFacet.sol(v1.0.0)
- src/Interfaces/ILayerSwapDepository.sol(v1.0.0)

## 4 Risk classification

| Severity level     | Impact: High | Impact: Medium | Impact: Low |
|--------------------|--------------|----------------|-------------|
| Likelihood: high   | Critical     | High           | Medium      |
| Likelihood: medium | High         | Medium         | Low         |
| Likelihood: low    | Medium       | Low            | Low         |

### 4.1 Impact

- High** leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
- Medium** global losses <10% or losses to only a subset of users, but still unacceptable.
- Low** losses will be annoying but bearable — applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

## 4.2 Likelihood

- High** almost certain to happen, easy to perform, or not easy but highly incentivized
- Medium** only conditionally possible or incentivized, but still relatively likely
- Low** requires stars to align, or little-to-no incentive

## 4.3 Action required for severity levels

- Critical** Must fix as soon as possible (if already deployed)
- High** Must fix (before deployment if not already deployed)
- Medium** Should fix
- Low** Could fix

## 5 Executive Summary

Over the course of 8 hours in total, [LI.FI](#) engaged with the [researcher](#) to audit the contracts described in section 3 of this document ("scope").

In this period of time a total of 5 issues were found.

| Project Summary |                                     |
|-----------------|-------------------------------------|
| Project Name    | LI.FI                               |
| Repository      | <a href="#">lifinance/contracts</a> |
| Commit          | <a href="#">0087bc33</a>            |
| Audit Timeline  | July 13, 2026 - July 15, 2026       |
| Methods         | Manual Review                       |
| Documentation   | High                                |
| Test Coverage   | High                                |

| Issues Found        |          |
|---------------------|----------|
| Critical Risk       | 0        |
| High Risk           | 0        |
| Medium Risk         | 1        |
| Low Risk            | 1        |
| Gas Optimizations   | 3        |
| Informational       | 0        |
| <b>Total Issues</b> | <b>5</b> |

## 6 Findings

### 6.1 Medium Risk

#### 6.1.1 Refunds and swap leftovers may be sent to a proxy or relayer

**Context:** [LayerSwapFacet.sol#L101](#), [LayerSwapFacet.sol#L128](#)

**Description:** Both excess native currency and pre-swap leftovers are returned to `msg.sender`. However, `msg.sender` may be Permit2Proxy or a relayer rather than the user.

Permit2Proxy does not forward returned assets after calling the Diamond. Assets retained by the proxy can only be withdrawn by its owner. Exact-output swap leftovers and excess `msg.value` can therefore become unavailable to the user or be delivered directly to a relayer.

**Recommendation:** Add an explicit `refundRecipient` to `LayerSwapData` and:

- Require it to be nonzero on both entrypoints.
- Include it in the EIP-712 signed payload.
- Pass it to `refundExcessNative`.
- Pass it as `_depositAndSwap`'s leftover receiver.

Add proxy-based tests for ERC20 leftovers and excess native refunds.

**LI.FI:** Fixed in [c4a7fb4](#)

**Researcher:** Verified fix.

### 6.2 Low Risk

#### 6.2.1 Mismatched swap output can drain the Diamond's bridge asset

**Context:** [LayerSwapFacet.sol#L139](#)

**Description:** The swap output is measured using the last swap's `receivingAssetId`, while `_startBridge` spends `bridgeData.sendingAssetId`. The facet never requires these assets to match.

A caller can therefore produce a cheap, high-decimal token through the swap while causing the same raw amount of another asset already held by the Diamond to be deposited into the caller's LayerSwap order.

**Recommendation:** Before `_depositAndSwap`, require the last swap's output asset to equal the signed bridge asset:

```
if (
    _swapData.length != 0 &&
    _swapData[_swapData.length - 1].receivingAssetId !=
    _bridgeData.sendingAssetId
) {
    revert InformationMismatch();
}
```

Add regression tests covering mismatched ERC20 and native output assets.

**LI.FI:** Fixed in [c5efa6c](#)

**Researcher:** Verified fix.

### 6.3 Gas Optimization

#### 6.3.1 Compact signatures can reduce calldata costs

**Context:** [LayerSwapFacet.sol#L65](#)

**Description:** The backend currently supplies a conventional 65-byte signature. Because signature is dynamically ABI-encoded, 65 bytes occupy three data words, while a 64-byte EIP-2098 compact signature occupies only two.

Solady already supports both 64-byte and 65-byte signatures. Using the compact representation therefore requires no contract ABI change and saves one padded calldata word, approximately 140 L1 calldata gas per call.

**Recommendation:** Update the backend signing pipeline to convert signatures to EIP-2098 compact form before encoding LayerSwapData. Retain support for conventional signatures for backward compatibility

**LI.FI:** Acknowledged, no contract change is needed.

**Researcher:** Acknowledged.

### 6.3.2 Constant EIP-712 values are hashed on every call

**Context:** [LayerSwapFacet.sol#L252](#)

**Description:** The domain separator hashes the constant name and version at runtime on every signature verification. This incurs two avoidable KECCAK256 operations, at least 72 execution gas plus memory allocation and writes.

**Recommendation:** Store the precomputed values as immutable variable in the constructor and update only if the `chain.id` gets updated.

**LI.FI:** Acknowledged, won't fix, the ~72 gas saved by pre-hashing the constant name/version is negligible.

**Researcher:** Acknowledged.

### 6.3.3 Calldata signature is unnecessarily copied into memory

**Context:** [LayerSwapFacet.sol#L239](#)

**Description:** `_layerSwapData.signature` already resides in calldata, but the facet calls `ECDSA.recover`, whose argument is a memory byte array. This causes the dynamic signature to be allocated and copied into memory before recovery.

Solady provides a calldata-specific recovery function that avoids this copy.

**Recommendation:** Use:

```
address recoveredSigner = ECDSA.recoverCalldata(
    digest,
    _layerSwapData.signature
);
```

**LI.FI:** Fixed in [ccdb37c](#)

**Researcher:** Verified fix.